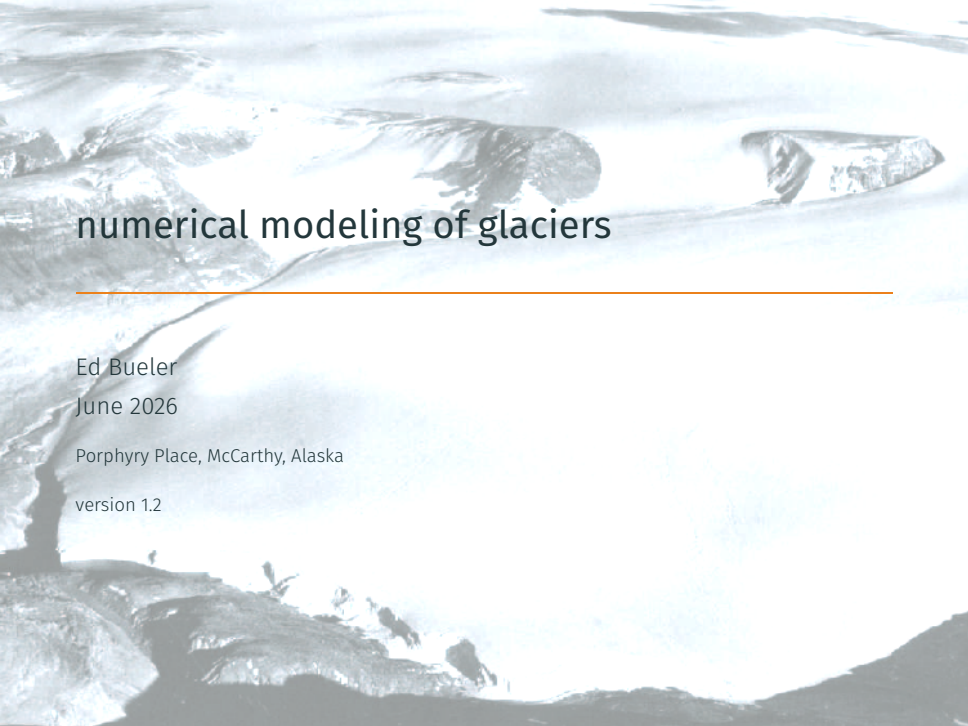




numerical modeling of glaciers

Ed Bueler

June 2026

Porphyry Place, McCarthy, Alaska

version 1.2

- please ask questions at any time!
- interruptions are encouraged

my slides, exercises, notes, and codes are hosted at
github.com/bueler/mccarthy

these slides: [slides/slides-2026.pdf](#)

notes: [notes/notes-2026.pdf](#)

exercises: [exercises/exercises-2026.pdf](#)

Python codes: [py/](#)

how does a glacier surface move? (hour 1)

basic mathematics

some numerics

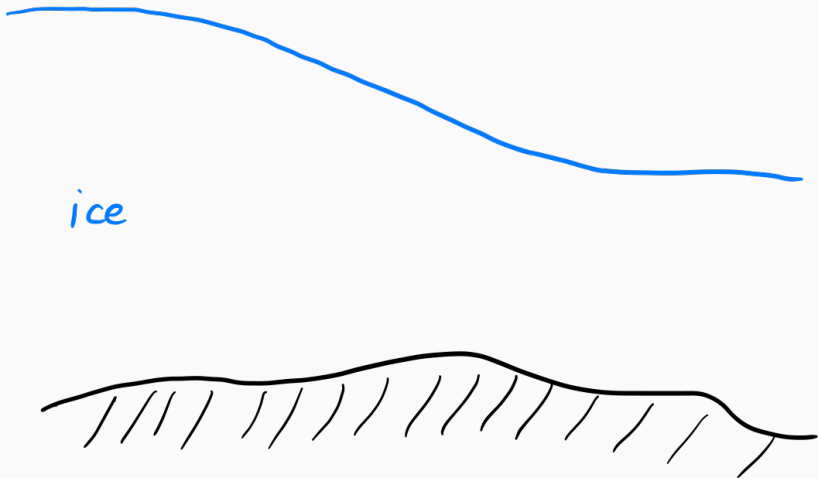
where does the velocity field come from? (hour 2)

basic mathematics

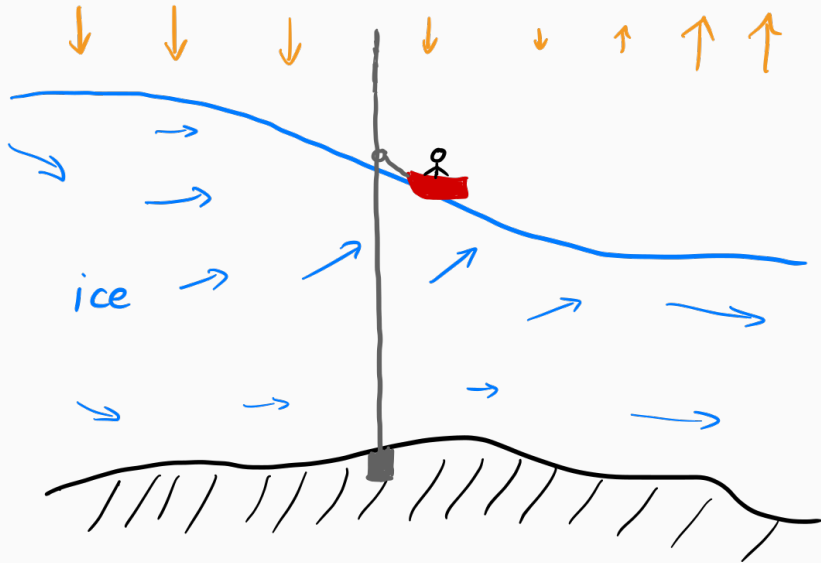
some numerics

how does a glacier surface move?

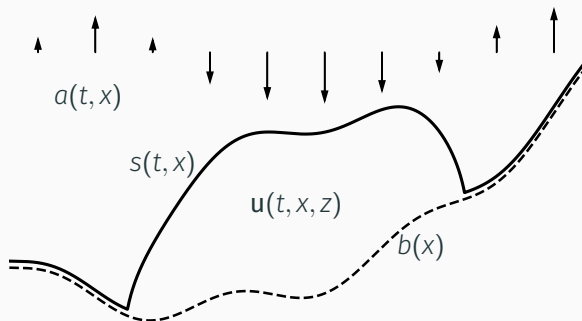
how does a glacier surface move?



how does a glacier surface move?

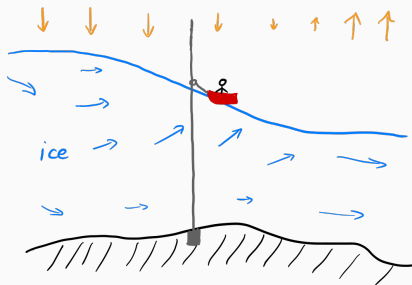


surface motion: planar (flow-line) notation



- 2D planar coordinates: x horizontal, z vertical
- $a(t, x)$: surface mass balance in ice-equivalent units (m s^{-1})
- $s(t, x)$: ice surface elevation (m)
- $\mathbf{u}(t, x, z) = \langle u(t, x, z), w(t, x, z) \rangle$: ice velocity (m s^{-1})
- $b(x)$: bed elevation (m)
- references for glacier equations: [GB09; SH13]

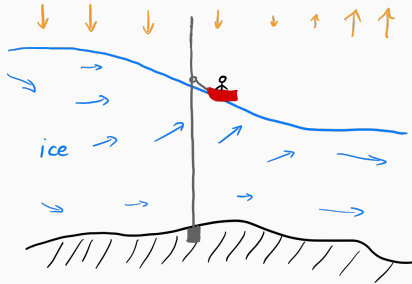
surface kinematical equation (SKE)



SKE: $\frac{\partial s}{\partial t} =$

the ice surface moves up and down according to:

surface kinematical equation (SKE)

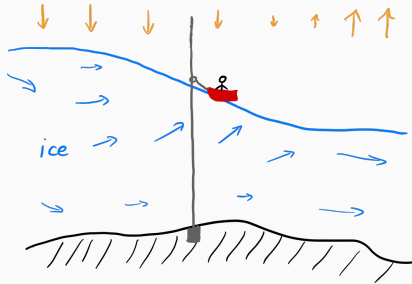


$$\text{SKE: } \frac{\partial s}{\partial t} = a$$

the ice surface moves up and down according to:

- surface mass balance (SMB)

surface kinematical equation (SKE)

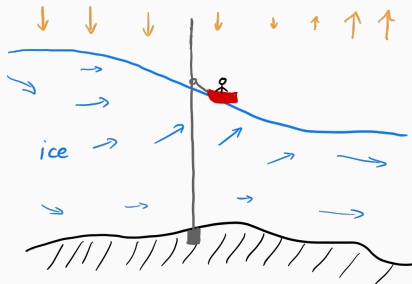


$$\text{SKE: } \frac{\partial s}{\partial t} = a + w$$

the ice surface moves up and down according to:

- surface mass balance (SMB)
- vertical component of ice velocity at surface

surface kinematical equation (SKE)

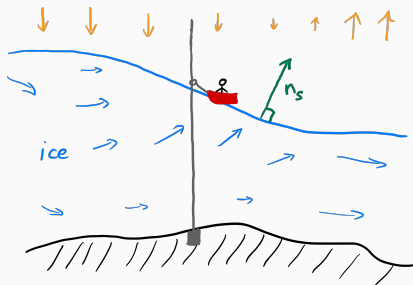


$$\text{SKE: } \frac{\partial s}{\partial t} = a + w - u \frac{\partial s}{\partial x}$$

the ice surface moves up and down according to:

- surface mass balance (SMB)
- vertical component of ice velocity at surface
- (horizontal component of ice velocity at surface) $\times$ (surface slope)

surface kinematical equation (SKE) in vector form



$$\text{SKE: } \frac{\partial s}{\partial t} - \mathbf{u} \cdot \mathbf{n}_s = a$$

- $\mathbf{u} = \langle u, w \rangle$ denotes the velocity **at the ice surface**
- $\mathbf{n}_s$ is a **surface-normal vector** which points upward:

$$\mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, 1 \right\rangle$$

- how would you prove the SKE?

answer: extra slides at end

plan for hour 1: SKE as a numerical glacier geometry model

- **model:** the planar surface kinematical equation (SKE)

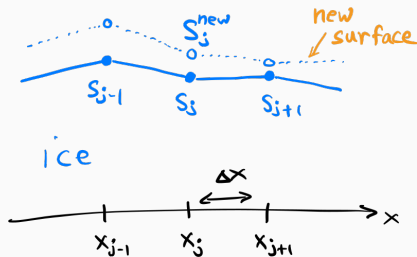
$$\frac{\partial s}{\partial t} - \mathbf{u} \cdot \mathbf{n}_s = a \quad \Longleftrightarrow \quad \frac{\partial s}{\partial t} + u \frac{\partial s}{\partial x} - w = a$$

- assume a, u, w are given functions/values
 - ...which is silly, but it is fixed in hour 2
- **numerical modeling goal:** build a computer program which evolves the glacier surface $z = s(t, x)$ according to the SKE
 - one short Python program in 1D: [py/surface.py](#)
- **to address:**
 1. how to compute time-dependent solutions?
 2. accuracy and stability?
 3. how to compute steady state solutions?

claim: the central object of glacier theory
is the surface kinematical equation

$$\frac{\partial s}{\partial t} - \mathbf{u} \cdot \mathbf{n}_s = a$$

discretize the SKE

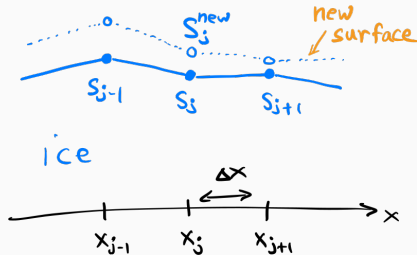


- planar (flow-line) SKE: $\frac{\partial s}{\partial t} + u \frac{\partial s}{\partial x} - w = a$
- choose time step $\Delta t > 0$ and grid spacing $\Delta x > 0$
- approximate partial derivatives by finite difference¹ quotients:

$$\frac{\partial s}{\partial t} \approx \frac{s_j^{new} - s_j}{\Delta t}, \quad \frac{\partial s}{\partial x} \approx \frac{s_{j+1} - s_{j-1}}{2\Delta x}$$

¹helpful textbooks include LeVeque [LeV07] and Morton & Mayers [MM05]

discretize the SKE



- the SKE becomes discrete:

$$\frac{s_j^{new} - s_j}{\Delta t} + u_j \frac{s_{j+1} - s_{j-1}}{2\Delta x} - w_j = a_j$$

- write as an **update** equation which determines s_j^{new} :

$$s_j^{new} = s_j - \Delta t u_j \frac{s_{j+1} - s_{j-1}}{2\Delta x} + \Delta t (a_j + w_j)$$

upwinding

- however, if we implement this explicit “forward-time centered-space” scheme

$$s_j^{\text{new}} = s_j - \Delta t u_j \frac{s_{j+1} - s_{j-1}}{2\Delta x} + \Delta t(a_j + w_j) \quad \leftarrow \text{bad}$$

then unstable things happen! (exercise 2)

- instead, it is known that an explicit **upwind** scheme is stable
 - **conditionally** stable, anyway, for this kind of **advection** problem
- in the case when the velocity is positive ($u_j \geq 0$) upwind is:

$$s_j^{\text{new}} = s_j - \Delta t u_j \frac{s_j - s_{j-1}}{\Delta x} + \Delta t(a_j + w_j) \quad \leftarrow \text{useful}$$

- general upwind formula: $\frac{\partial s}{\partial x} \approx \frac{1}{\Delta x} \begin{cases} (s_j - s_{j-1}), & u_j \geq 0 \\ (s_{j+1} - s_j), & u_j < 0 \end{cases}$
- the condition for stability is known (more soon)

implementation

- assuming $u_j \geq 0$, the upwind scheme for the SKE is

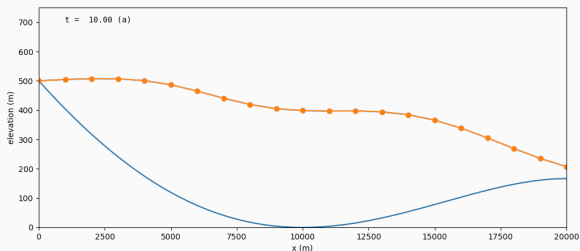
$$s_j^{\text{new}} = s_j - \Delta t u_j \frac{s_j - s_{j-1}}{\Delta x} + \Delta t (a_j + w_j)$$

- this becomes a Python function:

```
def explicitupwindstep(x, s, dt):  
    dx = x[1] - x[0]  
    snew = s.copy()  
    snew[1:] -= dt * u(x[1:]) * (s[1:] - s[:-1]) / dx  
    snew[1:] += dt * (a(x[1:]) + w(x[1:]))  
    return snew
```

- **x**, **s**, **snew** are 1D NumPy arrays
- **x** is an equally-spaced grid
- note that separate functions define $a(x)$, $u(x)$, $w(x)$
- note that **snew[0] = s[0]** is never changed; this is a boundary condition

live demo: evolution of glacier surface

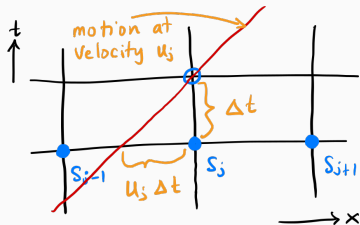


live demo

- run `py/surface.py`:
\$ `python3 surface.py`
- view images `output/frame*.png`
- however, the following modifications reveal stability issues:
 1. increase velocity: $u(x) \rightarrow 2.5 u(x)$
 2. lengthen time-steps: $\Delta t = 1 \text{ a} \rightarrow \Delta t = 2.5 \text{ a}$
 3. increase resolution: $\Delta x = 1000 \text{ m} \rightarrow \Delta x = 400 \text{ m}$

condition for time-stepping stability

- SKE: $\frac{\partial s}{\partial t} + u \frac{\partial s}{\partial x} - w = a$
- information about the surface travels at speed u
- from grid picture, the upwind scheme can only be stable if



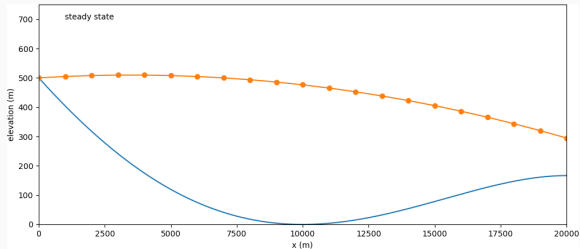
$$|u_j| \Delta t \leq \Delta x \quad \text{for every } j$$

- observed by Courant, Friedrichs, and Lewy in 1928
- rearranged, this is the **CFL condition** on the time step:

$$\Delta t \leq \frac{\Delta x}{\max |u_j|}$$

- exercise 12: add this to `surface.py` for $u(t, x)$

demo: steady state of a glacier surface

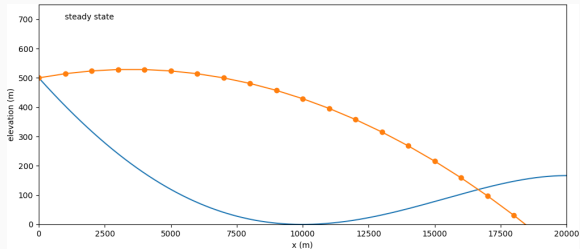


- SKE in steady-state ($\frac{\partial s}{\partial t} = 0$): $u \frac{\partial s}{\partial x} - w = a$

demo

- same run as before, but view `output/steady.png`
- the following modification reveals an **admissibility** issue:
 1. triple SMB: $a(x) \rightarrow 3a(x)$

demo: steady state of a glacier surface



- SKE in steady-state ($\frac{\partial s}{\partial t} = 0$): $u \frac{\partial s}{\partial x} - w = a$

demo

- same run as before, but view `output/steady.png`
- the following modification reveals an **admissibility** issue:
 1. triple SMB: $a(x) \rightarrow 3a(x)$

more advanced: 2D steady state of a glacier surface

- a surface elevation is **admissible** if $s \geq b$
- code: `py/surface2d.py`
 - this example uses **Firedrake**
 - solves 2D steady-state SKE:

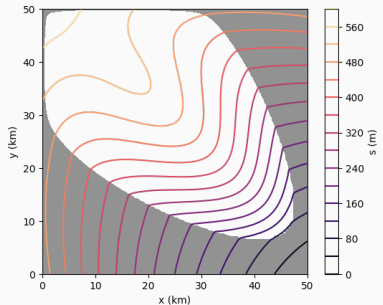
$$u \frac{\partial s}{\partial x} + v \frac{\partial s}{\partial y} - w = a$$

$$\text{or} \quad (u, v) \cdot \nabla s - w = a$$

$$\text{or} \quad -\mathbf{u} \cdot \mathbf{n}_s = a$$

$$\text{where } \mathbf{n}_s = \left\langle -\frac{\partial s}{\partial x}, -\frac{\partial s}{\partial y}, 1 \right\rangle$$

- result: $\rightarrow$



- the idea is that **SKE and $s \geq b$** yields the glacier outline and geometry, here in steady state with the SMB a
- this is the project 7 starting point

some questions when solving the SKE

modeling questions

1. whence the SMB a ?
2. whence the surface velocity u, w ?

topic of the 2nd hour!

mathematical questions

3. is the time-dependent SKE well-posed for suitable initial & boundary conditions?
4. is the steady-state SKE?
5. how do we understand admissibility ($s \geq b$) in the theory?

numerical questions

6. when the surface velocity u, w comes from a stress-balance submodel, is the upwind scheme still conditionally stable?
7. are there better explicit schemes?
8. is it better to use an implicit scheme?
9. how does one enforce admissibility in the right way?

where does the velocity field
come from? (hour 2)

ice in glaciers is a weird fluid

- if the ice were
 - faster-moving than it actually is, and
 - linearly-viscous

then it would be a “typical” fluid like liquid water

- for liquid water use the incompressible Navier-Stokes model:

$$\nabla \cdot \mathbf{u} = 0 \quad \text{incompressibility}$$

$$\rho \left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} \right) = -\nabla p + \nabla \cdot \boldsymbol{\tau}_{ij} + \rho \mathbf{g} \quad \text{stress balance}$$

$$2\nu(D\mathbf{u})_{ij} = \tau_{ij} \quad \text{flow law}$$

- $\mathbf{u}$ is velocity, p is pressure, $D\mathbf{u}$ is the strain rate tensor, $\boldsymbol{\tau}_{ij}$ is the deviatoric stress tensor, ρ is density, $\mathbf{g}$ is the acceleration of gravity
 - the stress balance equation is “ $ma = F$ ”, Newton’s 2nd law
- for such fluids: just look it up in computational fluid dynamics (CFD) textbooks, and use the software
- but that stuff is useless for glaciers ...

- numerical glacier modelling is first-class, large-scale CFD
 - ... but it's weird relative to atmosphere and ocean models
- consider what makes ocean flows exciting:
 - turbulence
 - convection
 - Coriolis force
 - density stratification
- none of the above list is relevant to ice flow, so CFD textbooks are of limited utility to glaciologists
- how do we model slow, cold, stiff, laminar, and inert ice?

ice is a slow, shear-thinning fluid

- ice fluid is **slow** and **non-Newtonian**

- “slow” is a technical term:

$$\rho(\mathbf{u}_t + \mathbf{u} \cdot \nabla \mathbf{u}) \approx 0 \quad \Longleftrightarrow \quad \left(\begin{array}{l} \text{forces of inertia} \\ \text{are neglected} \end{array} \right)$$

- ice is non-Newtonian in a shear-thinning way
 - viscosity ν is not constant, but depends on the fluid state
 - shear-thinning means higher strain rates implies lower viscosity
- the standard flow model is **Glen-Steinemann-Nye** (power-law) **Stokes**:

$$\nabla \cdot \mathbf{u} = 0 \quad \text{incompressibility}$$

$$0 = -\nabla p + \nabla \cdot \tau_{ij} + \rho \mathbf{g} \quad \text{stress balance}$$

$$D\mathbf{u}_{ij} = A\tau^{n-1}\tau_{ij} \quad \text{flow law}$$

- ρ is ice density, $\mathbf{g}$ is acceleration of gravity, A is ice softness
- the standard glacier geometry model is **SKE + Stokes**

regarding spatial derivatives

- the Stokes equations are loaded with spatial derivatives:

$$\nabla \cdot \mathbf{u} = 0$$

$$0 = -\nabla p + \nabla \cdot \boldsymbol{\tau}_{ij} + \rho \mathbf{g}$$

$$(D\mathbf{u})_{ij} = A\tau^{n-1}\tau_{ij}$$

- p is a scalar, $\mathbf{u}$ is vector, and $(D\mathbf{u})_{ij}, \tau_{ij}$ are tensors (matrices)
- by definition, in 3D coordinates (x_1, x_2, x_3) with $\mathbf{u} = \langle u_1, u_2, u_3 \rangle$:

$$\nabla p = \left\langle \frac{\partial p}{\partial x_1}, \frac{\partial p}{\partial x_2}, \frac{\partial p}{\partial x_3} \right\rangle \quad \text{vector}$$

$$\nabla \cdot \mathbf{u} = \frac{\partial u_1}{\partial x_1} + \frac{\partial u_2}{\partial x_2} + \frac{\partial u_3}{\partial x_3} \quad \text{scalar}$$

$$\nabla \cdot \boldsymbol{\tau}_{ij} = \langle \nabla \cdot \boldsymbol{\tau}_1, \nabla \cdot \boldsymbol{\tau}_2, \nabla \cdot \boldsymbol{\tau}_3 \rangle \quad \text{vector}$$

$$(D\mathbf{u})_{ij} = \frac{1}{2} (\nabla \mathbf{u} + (\nabla \mathbf{u})^\top)_{ij} \quad \text{tensor}$$

$$\tau^{n-1} = \left(\frac{1}{2} \tau_{ij} \tau_{ji} \right)^{(n-1)/2} \quad \text{scalar}$$

- getting comfortable with such notation is a skill in itself

“slow” means no memory of velocity/momentum

- note something simple: **no time derivatives** in the Stokes model!

$$\nabla \cdot \mathbf{u} = 0$$

$$0 = -\nabla p + \nabla \cdot \tau_{ij} + \rho \mathbf{g}$$

$$(D\mathbf{u})_{ij} = A\tau^{n-1}\tau_{ij}$$

- the velocity and pressure fields of a glacier are **instantaneous functions** of the glacier geometry and its boundary stresses
- thus a time-stepping ice sheet simulation will **recompute** the pressure and velocity fields **at every time step**
 - velocity from the previous time step is *not* required, unlike in a weather model for example
 - for glaciers, velocity is a “diagnostic” output, not needed for starting or restarting a model run

plane flow Stokes

- let's return to the x, z flow-line plane, and look at details
- the $n = 3$ Glen-law Stokes equations say:

$$u_x + w_z = 0 \quad \text{incompressibility}$$

$$0 = -p_x + \tau_{11,x} + \tau_{13,z} \quad \text{stress balance (x)}$$

$$0 = -p_z + \tau_{13,x} - \tau_{11,z} - \rho g \quad \text{stress balance (z)}$$

$$u_x = A\tau^2\tau_{11} \quad \text{flow law (diagonal)}$$

$$\frac{1}{2}(u_z + w_x) = A\tau^2\tau_{13} \quad \text{flow law (off-diagonal)}$$

- here the subscripts x, z denote partial derivatives
 - τ_{11} and τ_{33} are (deviatoric) longitudinal stresses
 - observation: $\tau_{33} = -\tau_{11}$ why?
 - τ_{13} is the “vertical” shear stress
 - after simplification: $\tau^2 = \tau_{11}^2 + \tau_{13}^2$
- we have 5 equations in 5 unknowns ($u, w, p, \tau_{11}, \tau_{13}$)
 - this is complicated enough ... try a simplified situation?

slab-on-a-slope

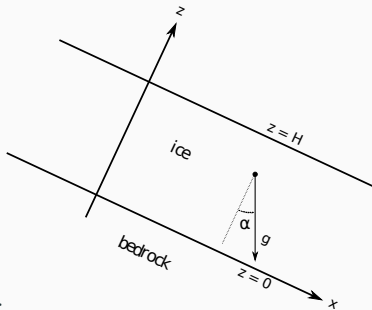
- suppose constant thickness
- suppose constant bedrock slope at angle α
- rotate the coordinates so x is bed-parallel

$$\mathbf{g} = g \sin \alpha \hat{x} - g \cos \alpha \hat{z}$$

- revised stress balance equations:

$$0 = -p_x + \tau_{11,x} + \tau_{13,z} + \rho g \sin \alpha$$

$$0 = -p_z + \tau_{13,x} - \tau_{11,z} - \rho g \cos \alpha$$



slab-on-a-slope

- revised equations:

$$u_x + w_z = 0$$

$$0 = -p_x + \tau_{11,x} + \tau_{13,z} + \rho g \sin \alpha$$

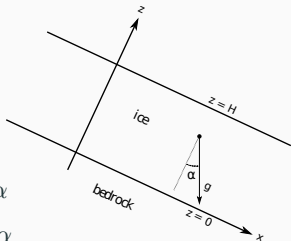
$$0 = -p_z + \tau_{13,x} - \tau_{11,z} - \rho g \cos \alpha$$

$$u_x = A\tau^2\tau_{11}$$

$$\frac{1}{2}(u_z + w_x) = A\tau^2\tau_{13}$$

- for **slab-on-a-slope** there is *no variation in x*: $\partial/\partial x = 0$
- the 5 equations simplify:

$w_z = 0$	$0 = \tau_{11}$
$\tau_{13,z} = -\rho g \sin \alpha$	$u_z = 2A\tau^2\tau_{13}$
$p_z = -\rho g \cos \alpha$	



slab-on-a-slope

- add non-penetrating basal velocity conditions:

$$u(0) = u_0, \quad w(0) = 0$$

- add zero stress conditions at the surface $z = H$:

$$p(H) = 0, \quad \tau_{13}(H) = 0$$

- then by integrating vertically, get:

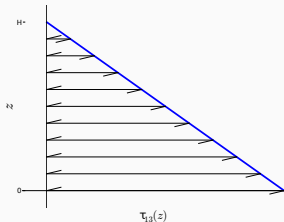
$$w(z) = 0$$

$$p(z) = \rho g \cos \alpha (H - z)$$

$$\tau_{13}(z) = \rho g \sin \alpha (H - z)$$

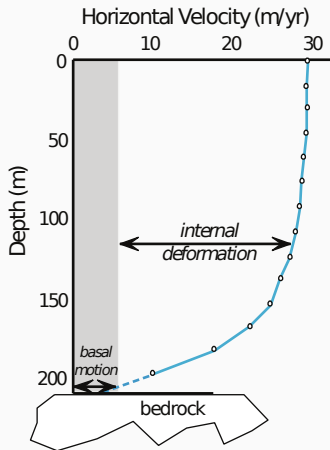
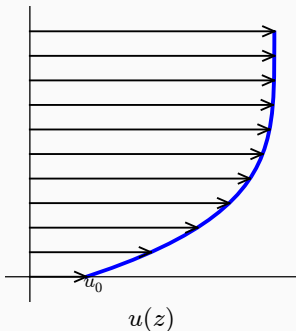
- note p & τ_{13} are linear in depth
- from $u_z = 2A\tau^2\tau_{13}$, integrate to get the **velocity formula**:

$$\begin{aligned} u(z) &= u_0 + 2A(\rho g)^3 (\sin \alpha)^3 \int_0^z (H - z')^3 dz' \\ &= u_0 + \frac{1}{2} A(\rho g)^3 (\sin \alpha)^3 (H^4 - (H - z)^4) \end{aligned}$$



slab-on-a-slope

- do we believe these results?
- velocity formula gives figure below
- compare to observations at right



[velocity profile of the Athabasca Glacier, Canada; from inclinometry (Savage & Paterson 1963)]

plan for hour 2: surface velocity from the SIA model

- **model:** the shallow ice approximation (SIA)
 - the SIA adopts the slab-on-slope formula for u as a *general formula for glaciers*
 - it is an imperfect stress balance (momentum) model, but it gives a meaningful velocity field in any geometry
 - there are careful shallowness scale analysis arguments for the SIA, using $\epsilon = [H]/[L] \rightarrow 0$ [Fow97]
- **numerical modeling goals:**
 - finite difference computations of surface velocity u, w
 - staggered grid points
 - one short Python code: [py/shallow.py](#)
- won't get to:
 - how does the SIA velocity field compare to the Stokes field for the same geometry?

project 8?

SIA for velocity

- the planar, isothermal, Glen-law, non-sliding **shallow ice approximation (SIA)** is the following system of equations:

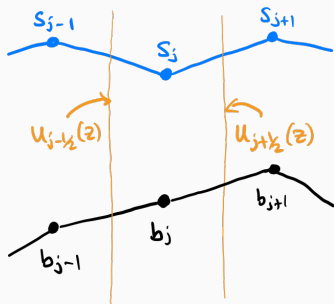
$$u = -\frac{2}{n+1}A(\rho g)^n \left[(s-b)^{n+1} - (s-z)^{n+1} \right] \left| \frac{\partial s}{\partial x} \right|^{n-1} \frac{\partial s}{\partial x}$$
$$\frac{\partial u}{\partial x} + \frac{\partial w}{\partial z} = 0$$

- u, w depend on t, x, z , but s depends only on t, x
- note that the horizontal velocity formula is of the form

$$u = -C \frac{\partial s}{\partial x},$$

with positive coefficient $C > 0$, so **ice flows downhill**

slopes and elevations onto the staggered grid



- center-differencing the surface elevations gives slopes computed on the staggered grid:

$$\left. \frac{\partial s}{\partial x} \right|_{j+1/2} = \frac{S_{j+1} - S_j}{\Delta x}$$

- averaging gives elevations computed on the staggered grid:

$$S_{j+1/2} = \frac{S_j + S_{j+1}}{2}, \quad b_{j+1/2} = \frac{b_j + b_{j+1}}{2}$$

Python code: finite differences for surface horizontal velocity

- recall the SKE needs only the *surface value* of velocity:

$$u_{j+1/2} = -C \left(\frac{H_j + H_{j+1}}{2} \right)^{n+1} \left| \frac{s_{j+1} - s_j}{\Delta x} \right|^{n-1} \frac{s_{j+1} - s_j}{\Delta x}$$

where $C = \frac{2}{n+1} A (\rho g)^n$ is constant, and $H = s - b$

- computing this at staggered-grid locations is a Python function:

```
def u(x, b, s):  
    dx = x[1] - x[0]  
    dsdx = (s[1:] - s[:-1]) / dx  
    Hstag = ((s[:-1] - b[:-1]) + (s[1:] - b[1:])) / 2.0  
    C_u = (2.0 / (n + 1)) * A * (rhoi * g) ** n  
    return -C_u * Hstag ** (n + 1) * abs(dsdx) ** (n - 1) * dsdx
```

surface vertical velocity (and ice flux)

- we can recover the vertical velocity w from **incompressibility**
 - recall: $\frac{\partial u}{\partial x} + \frac{\partial w}{\partial z} = 0$
 - integrate vertically, using non-penetrating ($w(t, x, b(x)) = 0$), then incompressibility, then Leibniz rule, then non-sliding ($u(t, x, b(x)) = 0$), to get w at the surface:

$$\begin{aligned} w(t, x, s(t, x)) &= - \int_{b(x)}^{s(t, x)} \frac{\partial u}{\partial x}(t, x, z) dz \\ &= u(x, s(t, x)) \frac{\partial s}{\partial x}(t, x) - \frac{\partial q}{\partial x}(t, x) \end{aligned}$$

- exercises 4 and 5 help with the Leibniz rule
- here $q(t, x) = \int_{b(x)}^{s(t, x)} u(t, x, z) dz$ is the **ice flux**
 - alternate form: $q = \bar{u}H$ where $\bar{u}$ is vertically-averaged horizontal velocity and H is ice thickness

ice flux and diffusion

- in the planar case the ice flux is $q = \int_b^s u \, dz$
- vertically-integrating the horizontal velocity gives

$$q = -D^{\text{SIA}} \frac{\partial s}{\partial x}$$

where the (effective) diffusivity is

$$D^{\text{SIA}} = \frac{2}{n+2} A (\rho g)^n H^{n+2} \left| \frac{\partial s}{\partial x} \right|^{n-1}$$

- the formula for surface vertical velocity also uses it:

$$w = u \frac{\partial s}{\partial x} + \frac{\partial}{\partial x} \left(D^{\text{SIA}} \frac{\partial s}{\partial x} \right)$$

- we will use this form to compute w at the surface
- we will return to what diffusivity *means*, but for now we apply finite differences to compute D^{SIA} and w in an example

finite differences for D^{SIA} and w

- for D^{SIA} on the staggered grid:

$$D_{j+1/2}^{\text{SIA}} = \frac{2}{n+2} A(\rho g)^n \left(\frac{H_j + H_{j+1}}{2} \right)^{n+2} \left| \frac{s_{j+1} - s_j}{\Delta x} \right|^{n-1}$$

def diffusiveD(x, b, s):

dx = x[1] - x[0]

dsdx = (s[1:] - s[:-1]) / dx

Hstag = ((s[:-1] - b[:-1]) + (s[1:] - b[1:])) / 2.0

C_D = (2.0 / (n + 2)) * A * (rhoi * g) ** n

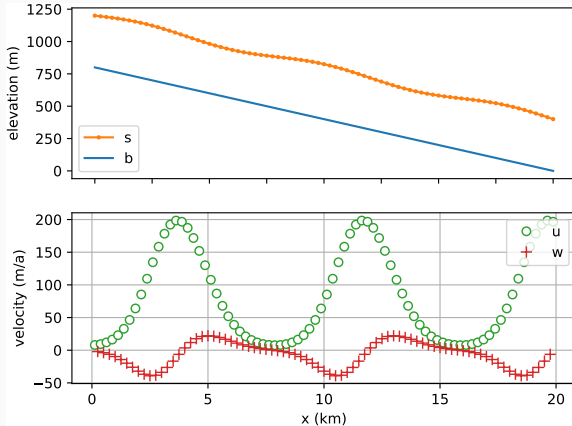
return C_D * Hstag ** (n + 2) * **abs**(dsdx) ** (n - 1)

- for w , let $\phi_{j+1/2} = u_{j+1/2} \frac{s_{j+1} - s_j}{\Delta x}$ on staggered grid, so:

$$w_j = \frac{\phi_{j-1/2} + \phi_{j+1/2}}{2} + \frac{D_{j+1/2}^{\text{SIA}}(s_{j+1} - s_j) - D_{j-1/2}^{\text{SIA}}(s_j - s_{j-1})}{\Delta x^2}$$

- we get w_j on the regular grid (Python function not shown)
- formulas for u , D^{SIA} , w are all in [py/shallow.py](#)

demo: SIA surface velocity for wavy surface



demo

- `$ python3 shallow.py`
- view image `output/uw.pdf`

time-dependent SKE + SIA model

- here are equations, so far, of the planar, non-sliding, and isothermal **time-dependent model** combining SKE and SIA:

$$\frac{\partial s}{\partial t} = a + w - u \frac{\partial s}{\partial x}$$

$$u = -\frac{2}{n+1} A(\rho g)^n (s-b)^{n+1} \left| \frac{\partial s}{\partial x} \right|^{n-1} \frac{\partial s}{\partial x} \quad (\text{surface value})$$

$$w = u \frac{\partial s}{\partial x} - \frac{\partial q}{\partial x} \quad (\text{surface value})$$

$$q = -D^{\text{SIA}} \frac{\partial s}{\partial x}$$

- we can simplify to just this for $s(t, x)$:

$$\frac{\partial s}{\partial t} = \frac{\partial}{\partial x} \left(D^{\text{SIA}} \frac{\partial s}{\partial x} \right) + a$$

- along with the formula: $D^{\text{SIA}} = \frac{2}{n+2} A(\rho g)^n H^{n+2} \left| \frac{\partial s}{\partial x} \right|^{n-1}$
- u, w, q are not directly needed to express geometry evolution

time-dependent SKE + SIA model as diffusion equation

- the SKE + SIA geometry evolution model is

$$\frac{\partial s}{\partial t} = \frac{\partial}{\partial x} \left(D^{\text{SIA}} \frac{\partial s}{\partial x} \right) + a$$

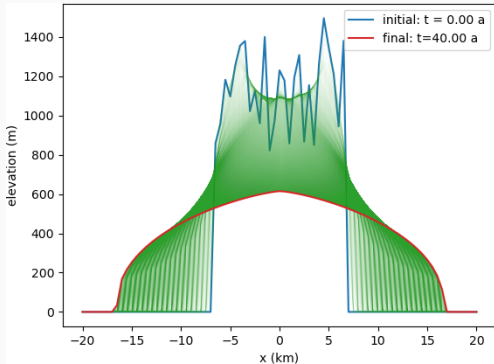
- this model **diffuses** the surface elevation
 - because ice flows downhill: $q = -D^{\text{SIA}} \frac{\partial s}{\partial x}$
- analogy to Fourier's heat equation:

$$\frac{\partial s}{\partial t} = \frac{\partial}{\partial x} \left(D^{\text{SIA}} \frac{\partial s}{\partial x} \right) + a \quad \leftrightarrow \quad \frac{\partial T}{\partial t} = \frac{\partial}{\partial x} \left(D \frac{\partial T}{\partial x} \right) + f$$

- T is temperature and f is a heat source
 - Fourier's law says heat flux is "downtemperature": $q = -D \frac{\partial T}{\partial x}$
- but the SKE + SIA is extremely nonlinear because

$$D^{\text{SIA}} = \frac{2}{n+2} A (\rho g)^n H^{n+2} \left| \frac{\partial s}{\partial x} \right|^{n-1}$$

demo: geometry evolution from a rough start



demo

- `$ python3 dome.py`
- view image `output/dome.png`

- the [notes](#) say much more about this time-dependent model

questions and issues for the SIA

modeling questions

1. what do we lose when using the SIA, versus Stokes?
2. how to add sliding to SIA?

mathematical issues

3. well-posedness not fully resolved
 - existence for the steady case
 - non-uniqueness in a some cases

numerical questions

4. how to control the numerical instabilities?
5. how to do efficient implicit time-stepping?

questions and issues for the SIA

modeling questions and answers

1. what do we lose when using the SIA, versus Stokes?
2. add sliding via a membrane-type stress balance

mathematical issues

3. well-posedness not fully resolved
 - existence for the steady case
 - non-uniqueness in a some cases

numerical questions and answers

4. use adaptive time-stepping to maintain stability (notes)
5. how to do efficient implicit time-stepping?

time-dependent SKE + Stokes

- what is the **standard model** of glaciology?
- **SKE + Stokes**, with admissible surfaces:

$$\frac{\partial s}{\partial t} - \mathbf{u} \cdot \mathbf{n}_s = a$$

$$\nabla \cdot \mathbf{u} = 0$$

$$0 = -\nabla p + \nabla \cdot \boldsymbol{\tau}_{ij} + \rho \mathbf{g}$$

$$D\mathbf{u}_{ij} = A\tau^{n-1}\tau_{ij}$$

$$s \geq b$$

- implementing well is a topic of current research
 - reliable and fast coupled solvers are not yet available

questions and issues for Stokes models

modeling questions

1. sliding laws determine the basal stress state; what's best?
2. how to couple to temperature/enthalpy equation?
3. how to adapt Stokes to compressible (e.g. temperate) ice?

mathematical questions

4. well-posedness of the Stokes model for fixed geometry?
 - essentially settled: yes [JR11]
5. well-posedness of the time-dependent standard model?
 - entirely open as far as I can tell; can't be easier than SIA case

numerical recommendation

6. as a practical matter, use a powerful finite element [ESW14] library
 - Firedrake, Elmer, FENiCs, Icepack, ...

an opinionated summary

- numerical models are unavoidable in glaciology
- most of your work is to understand the math, not to write code
 - even more true in the generative AI era

an opinionated summary

- numerical models are unavoidable in glaciology
- most of your work is to understand the math, not to write code
 - even more true in the generative AI era
- writing your own numerical models is of great value, but you should almost always toss the result in the trash once you understand it

an opinionated summary

- numerical models are unavoidable in glaciology
- most of your work is to understand the math, not to write code
 - even more true in the generative AI era
- writing your own numerical models is of great value, but you should almost always toss the result in the trash once you understand it
- surface mass/energy processes are as important as ice dynamics

an opinionated summary

- numerical models are unavoidable in glaciology
- most of your work is to understand the math, not to write code
 - even more true in the generative AI era
- writing your own numerical models is of great value, but you should almost always toss the result in the trash once you understand it
- surface mass/energy processes are as important as ice dynamics
- machine learning *after* understanding the laws of physics

an opinionated summary

- numerical models are unavoidable in glaciology
- most of your work is to understand the math, not to write code
 - even more true in the generative AI era
- writing your own numerical models is of great value, but you should almost always toss the result in the trash once you understand it
- surface mass/energy processes are as important as ice dynamics
- machine learning *after* understanding the laws of physics
- thanks so much for listening! questions?

further reading

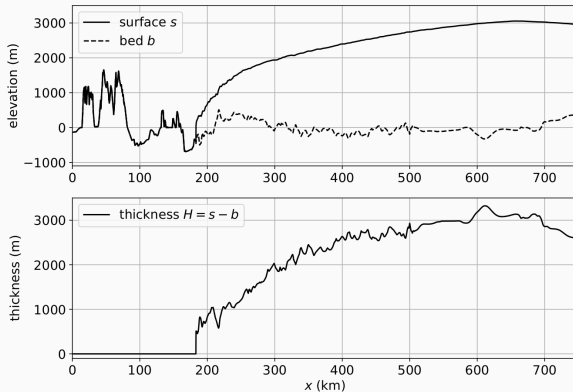
- [ESW14] H. C. Elman, D. J. Silvester, and A. J. Wathen. *Finite Elements and Fast Iterative Solvers: with Applications in Incompressible Fluid Dynamics*. 2nd. Oxford, UK: Oxford University Press, 2014.
- [Fow97] A. C. Fowler. *Mathematical Models in the Applied Sciences*. Cambridge Univ. Press, 1997.
- [GB09] R. Greve and H. Blatter. *Dynamics of Ice Sheets and Glaciers*. Berlin: Springer, 2009.
- [JR11] G. Jouvét and J. Rappaz. “Analysis and finite element approximation of a nonlinear stationary Stokes problem arising in glaciology”. In: *Advances in Numerical Analysis* (2011). DOI: [10.1155/2011/164581](https://doi.org/10.1155/2011/164581).
- [LeV07] R. J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM Press, 2007.
- [MM05] K. W. Morton and D. F. Mayers. *Numerical Solution of Partial Differential Equations: An Introduction*. 2nd. Cambridge University Press, 2005.
- [SH13] C. Schoof and I. J. Hewitt. “Ice-sheet dynamics”. In: *Annual Review of Fluid Mechanics* 45 (2013), pp. 217–239. DOI: [10.1002/jgrf.20146](https://doi.org/10.1002/jgrf.20146).

Extra Slides

1. why s is preferable to $H = s - b$
2. derive SKE from mass conservation

surface elevation versus thickness

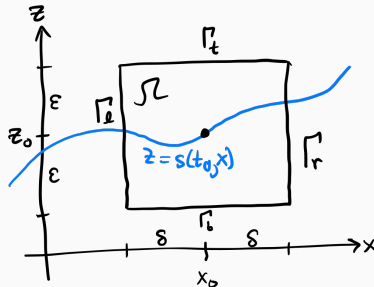
- why not use thickness $H = s - b$ to describe glacier geometry?



[Morlighem et al (2017); figure thanks to A. Aschwanden]

- answer: glacier surface s is generally smoother than the bedrock topography b
- H is rougher because $H = s - b$ inherits bed roughness

SKE from mass conservation 1

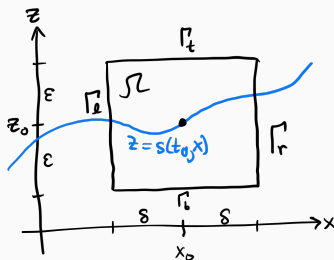


- $\Omega = (\text{rectangle centered at } (x_0, z_0)) = [x_0 - \delta, x_0 + \delta] \times [z_0 - \epsilon, z_0 + \epsilon]$
 - where $s(t_0, x_0) = z_0$
- let $M(t)$ be the mass of (constant-density) ice within Ω at time t :

$$M(t) = \int_{\Omega} \rho_i \mathbb{1}_{\{z < s(t, x)\}} dx dz = \rho_i \int_{x_0 - \delta}^{x_0 + \delta} s(t, x) - z_0 + \epsilon dx$$

- notation: ψ = mass flux, $\hat{n}$ = outward unit normal

SKE from mass conservation 2



- assumptions:
 - there are no mass sources within Ω
 - there is an interval of time t so that $z_0 - \epsilon < s(t, x) < z_0 + \epsilon$
 - the ice velocity $\mathbf{u}(t, x, z) = \langle u, w \rangle$ is defined on $\{z < s(t, x)\}$
 - the SMB $a(t, x)$ is a mass flux only across Γ_t :
 - $\psi|_{\Gamma_t} = -\rho_i a(t, x) \hat{\mathbf{z}}$ *note $a > 0$ is precipitation*
 - the other mass fluxes are kinematic (advective):
 - $\psi|_{\Gamma_b} = \rho_i w(t, x, z_0 - \epsilon) \hat{\mathbf{z}}$
 - $\psi|_{\Gamma_{\{\ell, r\}}} = \begin{cases} \rho_i u(t, x_{\{\ell, r\}}, z) \hat{\mathbf{x}}, & \text{if } z < s(t, x_{\{\ell, r\}}), \\ 0, & \text{otherwise} \end{cases}$

SKE from mass conservation 3

- mass is conserved:

$$\begin{aligned}\frac{dM}{dt} &= - \int_{\partial\Omega} \psi \cdot \hat{\mathbf{n}} \, ds \\ &= \underbrace{\rho_i \int_{x_0-\delta}^{x_0+\delta} a(t, x) \, dx}_{\Gamma_t} + \underbrace{\rho_i \int_{x_0-\delta}^{x_0+\delta} w(t, x, z_0 - \epsilon) \, dx}_{\Gamma_b} \\ &\quad + \underbrace{\rho_i \int_{z_0-\epsilon}^{s(t, x_0-\delta)} u(t, x_0 - \delta, z) \, dz}_{\Gamma_\ell} - \underbrace{\rho_i \int_{z_0-\epsilon}^{s(t, x_0+\delta)} u(t, x_0 + \delta, z) \, dz}_{\Gamma_r}\end{aligned}$$

SKE from mass conservation 4

- time derivative:

$$\begin{aligned}\frac{dM}{dt} &= \lim_{\omega \rightarrow 0} \frac{M(t + \omega) - M(t)}{\omega} \\&= \lim_{\omega \rightarrow 0} \frac{\rho_i}{\omega} \int_{\Omega} \mathbb{1}_{\{z < s(t+\omega, x)\}} - \mathbb{1}_{\{z < s(t, x)\}} dz dx \\&= \lim_{\omega \rightarrow 0} \frac{\rho_i}{\omega} \int_{x_0 - \delta}^{x_0 + \delta} s(t + \omega, x) - s(t, x) dx \\&= \rho_i \int_{x_0 - \delta}^{x_0 + \delta} \frac{\partial s}{\partial t}(t, x) dx\end{aligned}$$

- assume smoothness sufficient for Taylor expansions on Ω :

$$a(t, x) = a(t, x_0) + O(\delta)$$

$$u(t, x, z) = u(t, x_0, z_0) + O(\delta) + O(\epsilon)$$

$$w(t, x, z) = w(t, x_0, z_0) + O(\delta) + O(\epsilon)$$

$$\frac{\partial s}{\partial t}(t, x) = \frac{\partial s}{\partial t}(t, x_0) + O(\delta)$$

SKE from mass conservation 5

- combine mass conservation, time derivative, and Taylor:

$$\begin{aligned} 2\delta\rho_i \frac{\partial s}{\partial t}(t, x_0) + O(\delta^2) &= 2\delta\rho_i a(t, x_0) + O(\delta^2) \\ &\quad + 2\delta\rho_i w(t, x_0, z_0) + O(\delta^2) + O(\delta\epsilon) \\ &\quad - \rho_i u(t, x_0, z_0) \left[s(t, x_0 + \delta) - s(t, x_0 - \delta) \right] + O(\delta\epsilon) \end{aligned}$$

- divide by $2\rho_i\delta$:

$$\begin{aligned} \frac{\partial s}{\partial t}(t, x_0) &= a(t, x_0) + w(t, x_0, z_0) - u(t, x_0, z_0) \frac{s(t, x_0 + \delta) - s(t, x_0 - \delta)}{2\delta} \\ &\quad + O(\delta) + O(\epsilon) \end{aligned}$$

- limit $\delta, \epsilon \rightarrow 0$ to get SKE at $(t, x_0, z_0) = (t, x_0, s(t, x_0))$:

$$\frac{\partial s}{\partial t} = a + w - u \frac{\partial s}{\partial x}$$